

GNU Debugger (GDB)

Laboratórios de Informática III
Guião #5-2

Departamento de Informática
Universidade do Minho

Outubro de 2025

Contents

1	Introdução	2
2	Uso da ferramenta	2
2.1	Iniciar o GDB	2
2.2	Comandos básicos	2
2.3	Manipular pontos de paragem	3
3	Exemplo	3

1 Introdução

A *framework* GDB¹ (*GNU Debugger*) é uma poderosa ferramenta de depuração de programas em C, C++, e outras linguagens. É utilizada para auxiliar programadores a entenderem, analisarem, e corrigirem problemas nos seus programas. Fornece um ambiente interativo para examinar o estado de um programa em execução e é particularmente útil para localizar e corrigir erros de software, pois permite pausar a sua execução a qualquer momento para inspecionar variáveis, a pilha de chamadas (*call stack*), e outros aspetos do estado do programa.

Assim, algumas das suas principais funcionalidades são:

- **Execução linha-a-linha**, o que é fundamental para rastrear a execução e entender o comportamento do código.
- **Especificação de pontos de paragem** (*breakpoints*), quer numa função ou linha de código, facilitando a análise de uma parte específica do programa.
- **Inspeção do valor de variáveis** em qualquer ponto do programa. Tal é útil para verificar se as variáveis estão a ser atualizadas corretamente e se o programa está a executar conforme o esperado.

2 Uso da ferramenta

Para se utilizar a ferramenta GDB é necessário utilizar a *flag -g* no processo de compilação do código-fonte para ficheiros objeto, para incluir no executável gerado informações de depuração que permitem ao GDB (e outros *debuggers*) entender e exibir detalhes do código-fonte durante a depuração.

```
gcc -g [other flags] <source files> -o <programa>
```

2.1 Iniciar o GDB

Uma vez compilado o programa com informação necessária para ser depurado pelo GDB, podemos iniciá-lo, especificando o programa que pretende analisar:

```
gdb <program>
```

Ou caso seja necessário passar argumentos:

```
gdb --args <program> <args...>
```

2.2 Comandos básicos

Abaixo, listamos um conjunto de comandos básicos para utilizar o GDB e a sua utilidade:

- **run** (ou **r**) : Inicie a execução do programa.
- **kill** (ou **k**) : Termine a execução do programa.
- **continue** (ou **c**) : Continue a execução até ao próximo ponto de paragem (ou fim do programa).
- **step** (ou **s**) : Execute o programa linha a linha, entrando em funções chamadas.
- **next** (ou **n**) : Execute o programa linha a linha, mas não entre em funções chamadas.
- **print** (ou **p**) : Exibe o valor de uma variável. Por exemplo, `print var`.
- **backtrace** (ou **bt**) : Exibe a pilha de chamadas (*call stack*).
- **list** (ou **l**) : Exibe o código-fonte em torno do ponto atual.
- **quit** (ou **q**) : Sair do GDB.

¹ Algumas versões macOS não suportam a ferramenta GDB. Pode usar em alternativa a ferramenta LLDB cujo o funcionamento é idêntico (<https://lldb.llvm.org/use/map.html>).

2.3 Manipular pontos de paragem

A ferramenta GDB permite especificar pontos de paragem (*e.g.*, funções ou linhas de código) onde a execução do programa deve ser interrompida. Tal serve, por exemplo, para permitir ao programador verificar o valor de variáveis ou inspecionar a pilha de chamadas (*call stack*) naquele instante.

Assim, para manipular pontos de paragem podemos usar, entre outros, os seguintes comandos:

- `break` (ou `b`): Define um ponto de paragem. Por exemplo, `break nome_da_função` define um ponto de paragem no início de uma função. Ou `break programa.c:5` define um ponto de paragem na linha 5 do código-fonte de *programa.c*.
- `info breakpoints`: Exibe informações sobre os pontos de paragens definidos.
- `delete`: Remove um ponto de paragem. Por exemplo, `delete 1` remove o ponto de paragem com o número 1.
- `clear`: Limpa um ponto de paragem. Por exemplo, o comando `clear` num ponto de paragem remove-o.

Note ainda que o GDB não só interrompe a execução do programa quando encontra um ponto de paragem, mas também perante um problema na sua execução. Pois, quando um programa encontra um erro ou uma falha (como um “segmentation fault” ou “core dump”), o GDB pode ser usado para investigar o estado do programa nesse exato momento.

3 Exemplo

Vamos analisar um exemplo de uso da ferramenta GDB para deteção de erros. O código abaixo tem como objetivo dividir o valor da variável *x* pela variável *y* e imprimir o resultado.

```

1 //division.c
2 #include <stdio.h>
3
4 int main() {
5     int x = 5;
6     int y = 0;
7     int result = x / y;
8     printf("Resultado: %d\n", result);
9     return 0;
10 }
```

Ao compilar e executar o código acima, obtemos o seguinte erro:

```

1 $ gcc division.c -o division
2 $ ./division
3 > Floating point exception
```

Vamos utilizar o GDB para detetar o erro:

1. Compile o programa com a *flag* `-g`.

```

1 $ gcc -g division.c -o division
```

2. Inicie o ambiente do GDB para o nosso programa.

```

1 $ gdb ./division
```

3. Defina pontos de paragem. (Não seria necessário neste exemplo, apenas para demonstração vamos colocar um ponto de paragem na função *main* e outro na linha 6 do nosso código).

```

1 (gdb) break main
2 Breakpoint 1 at 0x652: file division.c, line 5.
3 (gdb) break division.c:6
4 Breakpoint 2 at 0x659: file division.c, line 6.

```

4. Inicie a execução do programa. O programa irá ser executado até encontrar um ponto de paragem ou um erro.

```

1 (gdb) run

```

5. Continuar a execução até à próxima paragem ou erro, iterativamente.

```

1 (gdb) continue

```

O programa eventualmente falhará devido à divisão por zero. O GDB mostrará informações de erro. Podemos ainda usar o comando `backtrace` para ver a pilha de chamadas e onde o erro ocorreu, e o comando `print` para verificar o valor das variáveis.

```

1 (gdb) continue
2 Program received signal SIGFPE, Arithmetic exception.
3 0x0000555555554664 in main () at division.c:7
4 7      int result = a / b;
5 (gdb) backtrace
6 #0 0x0000555555554664 in main () at division.c:7
7 (gdb) print a
8 $1 = 5
9 (gdb) print b
10 $2 = 0

```

Observando o resultado, podemos concluir que a origem do erro é na linha 7 do nosso programa, ou seja, quando tenta efetuar a divisão de 5 por 0, o que não é possível.

6. Uma vez terminada a depuração do nosso programa podemos sair.

```

1 (gdb) quit

```

Nota para depuramento de código em ambientes MacOS

Em sistemas *MacOS*, onde o `gdb` não se encontra disponível, poderá em alternativa utilizar a ferramenta `lldb` [1]. O seu funcionamento é idêntico ao da ferramenta `gdb`, havendo apenas diferenças nos nomes dos comandos a aplicar. Recomendamos a consulta deste resumo [2].

References

- [1] The LLDB Debugger
<https://lldb.llvm.org/>
- [2] LLDB cheat sheet
<https://github.com/carolanitz/DebuggingFun/blob/master/lldb%20cheat%20sheet.pdf>